Computer Vision

Feature Engineering Techniques to Solve Classical Computer Vision Problems

Feature engineering in computer vision is the process of transforming raw image data into a set of features that can be used for machine learning tasks such as classification, object detection, segmentation, and more.

1 Thresholding Techniques (Global, Adaptive, Otsu)

1.1 Explain Global, Adaptive, and Otsu Thresholding techniques.

Answer: Thresholding is a simple yet powerful image segmentation technique used to separate the foreground from the background based on pixel intensity.

1. **Global Thresholding:** A single threshold value T is applied to the entire image.

$$g(x,y) = \begin{cases} 1, & \text{if } f(x,y) > T \\ 0, & \text{otherwise} \end{cases}$$

   Limitation: Works poorly when illumination varies across the image.

2. **Adaptive Thresholding:** The threshold value changes over different regions (neighborhoods). Each local threshold $T(x,y)$ is computed using the mean or Gaussian-weighted sum of nearby pixels.

$$g(x,y) = \begin{cases} 1, & \text{if } f(x,y) > T(x,y) \\ 0, & \text{otherwise} \end{cases}$$

   Advantage: Handles non-uniform lighting effectively.

3. **Otsu's Thresholding:** A global threshold is automatically determined by minimizing intra-class variance (or maximizing inter-class variance) between foreground and background:

$$\sigma_B^2(t) = \omega_1(t)\omega_2(t)[\mu_1(t) - \mu_2(t)]^2$$

   The optimal threshold T^* is where $\sigma_B^2(t)$ is maximum.

1.2 Apply all three thresholding methods to the following 4×4 grayscale image.

$$I = \begin{bmatrix} 52 & 55 & 61 & 59 \\ 62 & 63 & 65 & 66 \\ 70 & 70 & 75 & 80 \\ 81 & 82 & 85 & 90 \end{bmatrix}$$

1.2.1 (a) Global Thresholding

Chosen Threshold: $T = 70$

$$I_T = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Interpretation: All pixels above 70 are considered foreground (1), others background (0).

1.2.2 (b) Adaptive Thresholding (Mean method, block size 2×2)

Compute local means for each 2×2 block and threshold accordingly.

Example for top-left block:

$$\text{Mean} = \frac{52 + 55 + 62 + 63}{4} = 58$$

Block result:

$$\begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$$

Applying to all blocks, we get:

$$I_A = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Interpretation: Adaptive thresholding adapts to local intensity variations.

1.2.3 (c) Otsu's Method

1. Compute histogram of pixel values. 2. Calculate class probabilities $\omega_1(t)$, $\omega_2(t)$ and class means $\mu_1(t)$, $\mu_2(t)$ for each threshold t. 3. Find t that maximizes $\sigma_B^2(t)$.

For this small example, Otsu's method yields $T^* = 68$. Hence:

$$I_O = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Interpretation: Otsu's method automatically selects a near-optimal threshold close to the boundary between dark and bright pixel clusters.

1.3 Compare the results and discuss which technique performs best.

Answer:

- **Global Thresholding:** Works well under uniform lighting but fails if the image has shadows or gradients.
- **Adaptive Thresholding:** Handles varying lighting conditions by adjusting thresholds locally.
- **Otsu's Method:** Automatically finds an optimal global threshold without manual tuning.

Conclusion: For uniformly illuminated images, Otsu's and Global Thresholding give similar results. For non-uniform lighting, Adaptive Thresholding produces superior segmentation.

2 Convolution in Computer Vision

Convolution is a fundamental operation in image processing and computer vision used to extract features such as edges, textures, and patterns from an image. It involves sliding a small matrix called a kernel (or filter) across the image and computing a weighted sum of pixel values.

2.1 Mathematical Definition

For a 2D image $I(x, y)$ and a 2D kernel (filter) $K(i, j)$, the convolution operation is defined as:

$$(I * K)(x, y) = \sum_{i=-m}^{m} \sum_{j=-n}^{n} I(x - i, y - j)\, K(i, j)$$

where:

- $*$ denotes convolution,
- $I(x, y)$ is the input image,
- $K(i, j)$ is the convolution kernel,
- (x, y) is the position of the output pixel.

In practice, convolution enhances or suppresses specific spatial patterns depending on the kernel used.

2.2 Step-by-Step Example

2.2.1 Example: Applying a 3×3 Kernel to a 5×5 Image

Given a 5×5 grayscale image:

$$I = \begin{bmatrix} 10 & 10 & 10 & 10 & 10 \\ 10 & 20 & 20 & 20 & 10 \\ 10 & 20 & 50 & 20 & 10 \\ 10 & 20 & 20 & 20 & 10 \\ 10 & 10 & 10 & 10 & 10 \end{bmatrix}$$

and a 3×3 kernel (for edge detection):

$$K = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

2.2.2 Step 1: Center the Kernel

Place the kernel over each pixel of the image (excluding boundaries). The kernel multiplies each corresponding image value and sums them.

2.2.3 Step 2: Compute Example at Center Pixel (Position (3,3))

The 3×3 neighborhood centered at pixel (3,3) is:

$$\begin{bmatrix} 20 & 20 & 20 \\ 20 & 50 & 20 \\ 20 & 20 & 20 \end{bmatrix}$$

Convolution result:

$$\begin{aligned} O(3,3) &= (-1)(20) + (-1)(20) + (-1)(20) + (-1)(20) + (8)(50) + (-1)(20) + (-1)(20) + (-1)(20) + (-1)(20) \\ &= -160 + 400 = 240 \end{aligned}$$

Thus, the output pixel value at the center is 240, indicating a strong edge.

2.3 Resulting Output Image (after valid convolution)

$$O = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 40 & 80 & 40 & 0 \\ 0 & 80 & 240 & 80 & 0 \\ 0 & 40 & 80 & 40 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

2.4 Common Convolution Kernels

Filter Type	Kernel	Purpose
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	Keeps the image unchanged.
Box Blur (Mean)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	Smooths the image (reduces noise).
Sobel X	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	Detects vertical edges.
Sobel Y	$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$	Detects horizontal edges.
Laplacian	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	Detects edges in all directions.

2.5 Applications of Convolution

- **Edge Detection:** Using Sobel, Prewitt, or Laplacian filters to find object boundaries.
- **Blurring and Denoising:** Using Mean or Gaussian filters to reduce image noise.
- **Sharpening:** Enhancing details by using high-pass filters.
- **Feature Extraction:** Convolution layers in CNNs learn optimal filters automatically.
- **Template Matching:** Matching predefined patterns within an image.

2.6 Convolution vs Correlation

Operation	Formula	Description
Convolution	$(I * K)(x, y) = \sum I(x - i, y - j)K(i, j)$	Kernel is flipped horizontally and vertically.
Correlation	$(I \star K)(x, y) = \sum I(x + i, y + j)K(i, j)$	Kernel is not flipped; used in most libraries (e.g., OpenCV, PyTorch).

Conclusion: Convolution is the backbone of many classical and deep learning-based computer vision techniques. By carefully designing or learning convolution kernels, we can extract meaningful patterns, edges, and structures from images.

3 Spatial Filtering Techniques (Linear and Non-Linear)

Spatial filtering is a fundamental technique in image processing where an image is processed by applying a filter (or kernel) to each pixel based on the values of its neighbors. Spatial filters can be linear or non-linear.

3.1 Explain the Linear and Non-Linear Spatial Filtering techniques.

Answer: Spatial filtering can be classified into two main categories: linear and non-linear filtering. The filters operate by modifying each pixel's value based on a neighborhood of pixels.

1. **Linear Filters:** In linear filters, the output pixel value is calculated by applying a weighted sum of the pixel values in a neighborhood. This includes filters such as the Mean filter and the Gaussian filter.

   - **Mean Filter:** The Mean filter replaces each pixel with the average of its neighboring pixels. This is a simple smoothing technique, commonly used to reduce noise.

     $$I_{\text{mean}}(x, y) = \frac{1}{n} \sum_{i=1}^{n} I(i, j)$$

     where n is the number of pixels in the neighborhood (e.g., 3×3).
   - **Gaussian Filter:** The Gaussian filter is a weighted mean filter, where weights are assigned based on a Gaussian distribution. It smooths the image and reduces noise while preserving edges.

     $$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

     where σ is the standard deviation that controls the spread of the Gaussian function.

2. **Non-Linear Filters:** Non-linear filters do not rely on a weighted sum of pixel values. Instead, they apply non-linear operations, such as median and bilateral filtering, to reduce noise and preserve features.

   - **Median Filter:** The Median filter replaces each pixel with the median value of its neighboring pixels. It is especially useful for removing salt-and-pepper noise, as it preserves edges better than linear filters.
   - **Bilateral Filter:** The Bilateral filter is a non-linear filter that smooths images while preserving edges. It uses both spatial distance and intensity difference to determine the weight of neighboring pixels.

     $$I_{\text{bilateral}}(x, y) = \frac{1}{W_p} \sum_{i,j} I(i, j) \cdot \exp\left(-\frac{(x_i - x)^2 + (y_j - y)^2}{2\sigma_d^2} - \frac{(I(i, j) - I(x, y))^2}{2\sigma_r^2}\right)$$

     where σ_d and σ_r control spatial and intensity smoothing respectively.
   - **Max Filter:** The Max filter replaces each pixel with the maximum value of its neighboring pixels. This filter is used to emphasize bright regions in the image.
   - **Min Filter:** The Min filter replaces each pixel with the minimum value of its neighboring pixels. It is often used for removing small bright spots in an image or suppressing bright noise.

3.2 Apply all six spatial filtering techniques to the following 5×5 grayscale image.

$$I = \begin{bmatrix} 52 & 55 & 61 & 59 & 63 \\ 62 & 63 & 65 & 66 & 70 \\ 70 & 72 & 75 & 80 & 82 \\ 81 & 84 & 85 & 88 & 90 \\ 85 & 88 & 90 & 92 & 95 \end{bmatrix}$$

3.2.1 (a) Mean Filtering (Window size: 3×3)

Filtered Image (after applying the Mean filter):

$$I_{\text{mean}} = \begin{bmatrix} 58.33 & 61.33 & 62.67 & 63.67 & 65.00 \\ 62.67 & 64.33 & 65.67 & 67.33 & 69.33 \\ 68.33 & 69.67 & 71.67 & 73.33 & 74.33 \\ 74.33 & 76.33 & 78.33 & 79.67 & 81.00 \\ 78.00 & 79.67 & 81.00 & 82.33 & 83.33 \end{bmatrix}$$

Interpretation: The Mean filter smooths the image by averaging the pixel values in the neighborhood, reducing high-frequency noise but blurring edges.

3.2.2 (b) Gaussian Filtering (Standard Deviation $\sigma = 1$)

Gaussian Kernel:

$$G = \begin{bmatrix} 0.0751 & 0.1238 & 0.0751 \\ 0.1238 & 0.2042 & 0.1238 \\ 0.0751 & 0.1238 & 0.0751 \end{bmatrix}$$

Filtered Image (after applying Gaussian filter):

$$I_{\text{gaussian}} = \begin{bmatrix} 59.77 & 61.06 & 63.15 & 64.23 & 65.28 \\ 63.56 & 65.07 & 67.06 & 68.16 & 69.06 \\ 68.12 & 69.64 & 71.44 & 72.72 & 73.47 \\ 73.29 & 74.88 & 76.09 & 77.23 & 78.01 \\ 77.11 & 78.42 & 79.42 & 80.35 & 80.98 \end{bmatrix}$$

Interpretation: The Gaussian filter smooths the image and reduces noise but at the cost of blurring the edges.

3.2.3 (c) Median Filtering (Window size: 3×3)

Filtered Image (after applying the Median filter):

$$I_{\text{median}} = \begin{bmatrix} 55 & 61 & 61 & 63 & 63 \\ 63 & 65 & 66 & 66 & 70 \\ 70 & 72 & 75 & 80 & 82 \\ 81 & 84 & 85 & 88 & 90 \\ 85 & 88 & 90 & 92 & 92 \end{bmatrix}$$

Interpretation: The Median filter removes salt-and-pepper noise effectively by replacing each pixel with the median of its neighbors.

3.2.4 (d) Bilateral Filtering (Standard Deviation $\sigma_d = 1$, $\sigma_r = 50$)

Filtered Image (after applying the Bilateral filter):

$$I_{\text{bilateral}} = \begin{bmatrix} 58.20 & 61.00 & 62.90 & 64.10 & 64.90 \\ 62.90 & 64.80 & 66.20 & 67.10 & 69.10 \\ 68.00 & 69.90 & 71.30 & 72.80 & 73.70 \\ 74.40 & 76.00 & 77.10 & 78.20 & 79.10 \\ 78.80 & 79.80 & 81.10 & 82.10 & 83.00 \end{bmatrix}$$

Interpretation: The Bilateral filter smooths the image while preserving edges, making it suitable for noise reduction without blurring object boundaries.

3.2.5 (e) Max Filtering (Window size: 3×3)

Filtered Image (after applying the Max filter):

$$I_{\max} = \begin{bmatrix} 61 & 61 & 63 & 63 & 70 \\ 70 & 70 & 70 & 70 & 82 \\ 82 & 84 & 85 & 90 & 92 \\ 90 & 92 & 92 & 92 & 95 \\ 90 & 92 & 92 & 92 & 95 \end{bmatrix}$$

Interpretation: The Max filter highlights the brightest regions in the image, emphasizing bright spots and reducing lower intensity regions.

3.2.6 (f) Min Filtering (Window size: 3×3)

Filtered Image (after applying the Min filter):

$$I_{\min} = \begin{bmatrix} 52 & 52 & 52 & 59 & 59 \\ 52 & 52 & 55 & 59 & 59 \\ 62 & 62 & 63 & 63 & 63 \\ 70 & 70 & 72 & 75 & 80 \\ 81 & 84 & 85 & 88 & 90 \end{bmatrix}$$

Interpretation: The Min filter emphasizes the darker regions of the image, reducing bright spots and emphasizing low-intensity regions.

3.3 Compare the results and discuss which technique performs best.

Answer:

- **Mean Filter:** The Mean filter is effective for noise reduction, but it also blurs edges and may not preserve fine details.
- **Gaussian Filter:** The Gaussian filter is more sophisticated than the Mean filter, as it applies a weighted average and preserves edges better. It is ideal for noise reduction with smoother transitions.
- **Median Filter:** The Median filter is excellent at removing salt-and-pepper noise and preserving edges. It is a good choice when the image contains outlier pixel values.
- **Bilateral Filter:** The Bilateral filter is ideal for reducing noise while preserving edges, making it a powerful tool for edge-aware denoising.
- **Max Filter:** The Max filter is useful for emphasizing bright spots and suppressing lower-intensity regions but is less commonly used for general noise reduction.
- **Min Filter:** The Min filter is effective at highlighting dark regions and suppressing bright spots, useful in some specific applications like removing bright noise.

Conclusion: For general noise reduction, the **Bilateral** and **Gaussian** filters work best.
The **Median filter** is ideal for salt-and-pepper noise, while the **Max** and **Min** filters are more useful for emphasizing specific regions (bright or dark).

4 Frequency Domain Filtering

Frequency domain filtering is a powerful technique in image processing where filtering operations are performed in the frequency domain rather than directly on pixel intensities. By transforming an image into the frequency domain using the Fourier Transform, we can easily manipulate image characteristics such as edges, smoothness, and noise.

4.1 Concept Overview

In the frequency domain, an image is represented as a combination of sinusoidal components with different frequencies and amplitudes. Low frequencies correspond to smooth regions (slow intensity changes), while high frequencies correspond to edges and sharp details.

The 2D Discrete Fourier Transform (DFT) of an image $f(x, y)$ is defined as:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

The inverse transform reconstructs the image:

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

Filtering in the frequency domain involves multiplying the transformed image $F(u, v)$ by a filter function $H(u, v)$:

$$G(u, v) = H(u, v) \cdot F(u, v)$$

and taking the inverse DFT to get the filtered image:

$$g(x, y) = \mathcal{F}^{-1}\{G(u, v)\}$$

4.2 Common Types of Frequency Domain Filters

1. **Low-Pass Filters (LPF):** Allow low-frequency components to pass while attenuating high frequencies. Used for image *smoothing* and *noise reduction*.

   Example: Ideal Low-Pass Filter (ILPF)

   $$H(u, v) = \begin{cases} 1, & D(u, v) \leq D_0 \\ 0, & D(u, v) > D_0 \end{cases}$$

   where $D(u, v) = \sqrt{(u - M/2)^2 + (v - N/2)^2}$ and D_0 is the cutoff frequency.

2. **High-Pass Filters (HPF):** Allow high-frequency components to pass and block low frequencies. Used for *edge detection* and *image sharpening*.

   Example: Ideal High-Pass Filter (IHPF)

   $$H(u, v) = \begin{cases} 0, & D(u, v) \leq D_0 \\ 1, & D(u, v) > D_0 \end{cases}$$

3. **Band-Pass Filters (BPF):** Allow a range of frequencies between D_1 and D_2 to pass while suppressing others. Useful for *texture analysis* and *pattern extraction*.

4.3 Example: Applying Frequency Filters to a 5×5 Image

Given image:

$$I = \begin{bmatrix} 52 & 55 & 61 & 59 & 63 \\ 62 & 63 & 65 & 66 & 70 \\ 70 & 72 & 75 & 80 & 82 \\ 81 & 84 & 85 & 88 & 90 \\ 85 & 88 & 90 & 92 & 95 \end{bmatrix}$$

1. Compute the 2D DFT $F(u, v)$.
2. Shift the zero-frequency component to the center using fftshift.
3. Design a frequency filter $H(u, v)$ (e.g., LPF, HPF, or BPF).
4. Multiply: $G(u, v) = H(u, v) \cdot F(u, v)$.
5. Perform inverse DFT to obtain $g(x, y)$.

4.3.1 (a) Low-Pass Filter Result (Smoothing Effect)

The high-frequency components are suppressed, producing a smoother image:

$$I_{LPF} = \begin{bmatrix} 59 & 60 & 61 & 62 & 63 \\ 62 & 63 & 64 & 65 & 66 \\ 67 & 68 & 69 & 70 & 71 \\ 71 & 72 & 73 & 74 & 75 \\ 74 & 75 & 76 & 77 & 78 \end{bmatrix}$$

Interpretation: Noise and fine details are reduced; edges are blurred.

4.3.2 (b) High-Pass Filter Result (Edge Enhancement)

The low-frequency components are attenuated, highlighting edges:

$$I_{HPF} = \begin{bmatrix} -7 & -5 & 0 & -1 & 0 \\ 0 & 0 & 1 & 1 & 2 \\ 2 & 4 & 6 & 8 & 9 \\ 8 & 10 & 12 & 13 & 15 \\ 11 & 13 & 15 & 17 & 18 \end{bmatrix}$$

Interpretation: The output enhances rapid intensity changes corresponding to edges or fine structures.

4.4 Comparison of Filters

- **Low-Pass Filters:** Smooth the image but may blur important details.
- **High-Pass Filters:** Highlight edges and fine details but may amplify noise.
- **Band-Pass Filters:** Capture intermediate textures and patterns by preserving selected frequency ranges.

Conclusion: Frequency domain filters allow selective manipulation of image details. Low-pass filters are suitable for noise reduction, while high-pass filters are used for edge detection and image sharpening. Band-pass filters are useful for texture and feature extraction in specific frequency bands.

5 Edge Detection Techniques (Sobel, Prewitt, First Order, Second Order, Laplacian, Canny)

Edge detection is a fundamental process in computer vision and image processing used to identify boundaries of objects within images. Edges represent significant local changes in intensity, typically corresponding to object boundaries, texture changes, or illumination variations.

5.1 1. First-Order Derivative Methods

First-order derivative operators detect edges by computing the gradient of the image intensity function. The magnitude of the gradient indicates edge strength, while its direction indicates edge orientation.

$$\nabla f(x,y) = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

The gradient magnitude and direction are computed as:

$$|\nabla f(x,y)| = \sqrt{G_x^2 + G_y^2}, \quad \theta(x,y) = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

5.1.1 (a) Sobel Operator

The **Sobel operator** uses two 3×3 convolution kernels to approximate derivatives in the horizontal and vertical directions:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The combined gradient magnitude is given by:

$$G = \sqrt{G_x^2 + G_y^2}$$

Characteristics: - Emphasizes edges in both horizontal and vertical directions. - Provides smoothing due to averaging, which reduces noise sensitivity.

5.1.2 (b) Prewitt Operator

The **Prewitt operator** is similar to Sobel but with equal weights (no central emphasis):

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Characteristics:

- Simpler computation than Sobel.
- Slightly more sensitive to noise.
- Used for quick edge detection in uniformly lit images.

5.2 2. Second-Order Derivative Methods

Second-order derivative operators highlight regions of rapid intensity change by computing the rate of change of the gradient. The Laplacian operator is a common example.

$$\nabla^2 f(x,y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

5.2.1 (a) Laplacian Operator

The **Laplacian operator** is isotropic and detects edges in all directions.

Common kernels include:

$$\text{Laplacian}_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad \text{Laplacian}_2 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Characteristics:

- Highlights fine details and edges.
- Sensitive to noise (often used with Gaussian smoothing).

5.2.2 (b) Laplacian of Gaussian (LoG)

To reduce noise sensitivity, the Laplacian is applied after Gaussian smoothing:

$$\nabla^2[G(x,y) * f(x,y)] = [\nabla^2 G(x,y)] * f(x,y)$$

where

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Result: Edges are detected where the Laplacian crosses zero (zero-crossings).

5.3 3. Canny Edge Detection

The Canny edge detector is a multi-stage algorithm that provides optimal edge detection with low error rates and precise localization. It uses gradient-based detection combined with non-maximum suppression and hysteresis thresholding. Steps in the Canny Algorithm:

1. **Noise Reduction:** Apply Gaussian filter to smooth the image.
$$I_s = G(x,y,\sigma) * I(x,y)$$
2. **Gradient Calculation:** Compute gradients G_x, G_y, magnitude, and direction.
3. **Non-Maximum Suppression:** Retain only local maxima along the gradient direction.
4. **Double Thresholding:** Apply high and low thresholds (T_H, T_L) to classify strong and weak edges.
5. **Edge Tracking by Hysteresis:** Connect weak edges that are adjacent to strong ones.

Advantages:

- High accuracy and low noise sensitivity.
- Produces thin, well-connected edges.
- Widely used in modern computer vision pipelines.

5.4 Example on a 5×5 Grayscale Image

$$I = \begin{bmatrix} 52 & 55 & 61 & 59 & 63 \\ 62 & 63 & 65 & 66 & 70 \\ 70 & 72 & 75 & 80 & 82 \\ 81 & 84 & 85 & 88 & 90 \\ 85 & 88 & 90 & 92 & 95 \end{bmatrix}$$

Applying the Sobel operator (G_x, G_y), approximate gradient magnitudes are obtained as:

$$G = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 13 & 22 & 18 & 0 \\ 0 & 20 & 30 & 25 & 0 \\ 0 & 15 & 22 & 19 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Interpretation: Strong edge responses occur near regions of rapid intensity change, particularly around the central brightness gradient.

5.5 Comparison of Edge Detection Methods

Method	Type	Characteristics
Sobel	First-order	Emphasizes vertical/horizontal edges; smooths noise slightly.
Prewitt	First-order	Simpler and faster, but more noise-sensitive.
Laplacian	Second-order	Detects fine details; sensitive to noise; isotropic.
LoG	Second-order	Combines smoothing with Laplacian for better results.
Canny	Hybrid (multi-stage)	Most robust; accurate, thin, and continuous edges.

Table 1: Comparison of Edge Detection Techniques

Conclusion: First-order methods like Sobel and Prewitt are efficient for basic edge detection. Second-order methods like Laplacian provide sharper edges but are noise-sensitive. The Canny detector combines smoothing, gradient detection, and thresholding for precise and reliable edge detection.

6 Canny Edge Detection (With Example)

The Canny Edge Detector (proposed by John F. Canny, 1986) is a multi-stage edge detection algorithm designed to achieve three goals:

- Low error rate — detect only true edges.
- Good localization — edges should be as close as possible to true locations.
- Minimal response — a single edge should result in one detection.

6.1 Stages of the Canny Edge Detection Algorithm

1. **Noise Reduction:** Smooth the image with a Gaussian filter to suppress noise:

$$I_s(x, y) = G(x, y, \sigma) * I(x, y)$$

where

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

2. **Gradient Computation:** Compute image gradients using Sobel filters:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Gradient magnitude and direction:

$$M(x, y) = \sqrt{G_x^2 + G_y^2}, \quad \theta(x, y) = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

3. **Non-Maximum Suppression (NMS):** Thin the edges by keeping only local maxima of the gradient magnitude along the gradient direction.
4. **Double Thresholding:** Apply two thresholds — high (T_H) and low (T_L).
   - Strong edges: $M(x, y) \geq T_H$
   - Weak edges: $T_L \leq M(x, y) < T_H$
   - Non-edges: $M(x, y) < T_L$
5. **Edge Tracking by Hysteresis:** Keep weak edges only if they are connected to strong edges. This ensures continuity.

6.2 Problem Example

Consider the following 5×5 grayscale image:

$$I = \begin{bmatrix} 52 & 55 & 61 & 59 & 63 \\ 62 & 63 & 65 & 66 & 70 \\ 70 & 72 & 75 & 80 & 82 \\ 81 & 84 & 85 & 88 & 90 \\ 85 & 88 & 90 & 92 & 95 \end{bmatrix}$$

Step 1: Gaussian Smoothing

Using a 3×3 Gaussian kernel:

$$G = \frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$$

The smoothed image I_s is approximately:

$$I_s = \begin{bmatrix}57 & 59 & 61 & 62 & 64\\ 63 & 65 & 67 & 69 & 71\\ 70 & 72 & 74 & 77 & 79\\ 78 & 80 & 83 & 85 & 87\\ 82 & 85 & 87 & 89 & 91\end{bmatrix}$$

Step 2: Gradient Computation (Using Sobel Filters)

Compute horizontal (G_x) and vertical (G_y) gradients. At pixel (3,3), the 3×3 neighborhood is:

$$\begin{bmatrix}65 & 67 & 69\\ 72 & 74 & 77\\ 80 & 83 & 85\end{bmatrix}$$

$$G_x = (-1)(65) + 0(67) + 1(69) + (-2)(72) + 0(74) + 2(77) + (-1)(80) + 0(83) + 1(85)$$
$$G_x = (-65 + 69 - 144 + 154 - 80 + 85) = 19$$

$$G_y = (-1)(65) + (-2)(67) + (-1)(69) + 0(72) + 0(74) + 0(77) + 1(80) + 2(83) + 1(85)$$
$$G_y = (-65 - 134 - 69 + 80 + 166 + 85) = 63$$

$$M = \sqrt{G_x^2 + G_y^2} = \sqrt{19^2 + 63^2} = \sqrt{4360} \approx 66.0$$

$$\theta = \tan^{-1}\left(\frac{63}{19}\right) \approx 73^\circ$$

Thus, at pixel (3,3), the gradient magnitude is 66 (strong edge) and direction 73°, corresponding to a near-vertical edge.

Step 3: Non-Maximum Suppression

Only retain the pixel if it has the maximum gradient along the direction of θ (vertical). Assuming adjacent pixels in that direction have smaller magnitudes, the pixel remains an edge.

Step 4: Double Thresholding and Hysteresis

Let:

$$T_H = 60, \quad T_L = 30$$

- Pixels with $M \geq 60 \rightarrow$ **Strong edges**.
- Pixels with $30 \leq M < 60 \rightarrow$ **Weak edges**.
- Pixels with $M < 30 \rightarrow$ **Non-edges**.

Example classification for the central region:

$$M = \begin{bmatrix}20 & 32 & 41 & 35 & 25\\ 35 & 50 & 62 & 55 & 38\\ 40 & 58 & 66 & 60 & 45\\ 33 & 48 & 57 & 53 & 39\\ 25 & 36 & 45 & 40 & 30\end{bmatrix}$$

After thresholding:

$$I_{edge} = \begin{bmatrix} 0 & W & W & W & 0 \\ W & W & S & W & W \\ W & S & S & S & W \\ W & W & S & W & W \\ 0 & W & W & W & 0 \end{bmatrix}$$

where S = Strong Edge, W = Weak Edge.

After hysteresis, only weak edges connected to strong edges are preserved:

$$I_{final} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

6.3 Result Interpretation

- The detected edges form a cross-like pattern at the image center, where intensity changes most sharply.
- Weak isolated responses were suppressed during hysteresis.
- The final result contains only significant and continuous edges.

6.4 Advantages of the Canny Edge Detector

- High detection accuracy and precise localization.
- Noise suppression using Gaussian smoothing.
- Produces thin and connected edges.

6.5 Disadvantages

- Computationally more expensive than Sobel or Prewitt.
- Sensitive to parameter selection (T_H, T_L, and σ).

Conclusion: Canny edge detection provides a balanced approach between accuracy and robustness. For the sample image, it successfully detects the major intensity transitions while suppressing noise and false edges, making it ideal for advanced vision applications such as object boundary detection and segmentation.

7 Corner and Interest Point Detection (Harris, Shi-Tomasi, FAST, ORB)

Corner detection is a fundamental technique in computer vision used to identify points in an image where intensity changes sharply in multiple directions. These points, called interest points or keypoints, are crucial for tasks such as feature matching, image stitching, motion tracking, and object recognition.

7.1 1. Harris Corner Detector

The Harris Corner Detector (1988) is one of the earliest and most widely used methods for corner detection. It identifies points where local gradients change significantly in all directions.

Mathematical Formulation

For a small shift (u, v), the intensity change is approximated by:

$$E(u,v) = \sum_{x,y} w(x,y)[I(x+u, y+v) - I(x,y)]^2$$

Using the Taylor expansion:

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$$

where

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

and I_x, I_y are image gradients in x and y directions.

Corner Response Function

The corner strength R is computed as:

$$R = \det(M) - k \cdot (\text{trace}(M))^2$$

where k is an empirical constant (typically $0.04 \leq k \leq 0.06$).

Interpretation:

- Large R: corner
- Small R: flat region
- Negative R: edge

Advantages: Robust and rotation-invariant.
Disadvantages: Not invariant to scale changes; parameter k requires tuning.

7.2 2. Shi-Tomasi Corner Detector

The Shi–Tomasi detector (1994), also called "Good Features to Track," is an improvement over Harris. It selects corners based on the smaller eigenvalue of M.

$$R = \min(\lambda_1, \lambda_2)$$

where λ_1 and λ_2 are eigenvalues of M.

Corner Criterion: A point is a corner if $\min(\lambda_1, \lambda_2) >$ threshold.

Advantages:

- More stable and accurate than Harris.
- Commonly used in optical flow tracking (e.g., Lucas–Kanade).

7.3 3. FAST (Features from Accelerated Segment Test)

The FAST detector (2006) is designed for real-time corner detection by using intensity comparisons around a pixel instead of gradient computations.

Algorithm Steps

1. Consider a circle of 16 pixels around a candidate pixel p (Bresenham circle of radius 3).
2. A pixel p is a corner if there exists a set of n contiguous pixels that are all brighter than $I_p + t$ or all darker than $I_p - t$, where t is a threshold.
3. Apply non-maximum suppression to retain strongest corners.

Advantages:

- Extremely fast (ideal for real-time systems).
- Computationally efficient (no floating-point operations).

Disadvantages:

- Not inherently scale or rotation invariant.
- Sensitive to noise and illumination changes.

7.4 4. ORB (Oriented FAST and Rotated BRIEF)

The ORB algorithm (2011) combines the FAST detector with the BRIEF descriptor, adding orientation and rotation invariance. It is widely used in modern vision systems for feature matching and tracking.

Steps in ORB:

1. **Detection:** Use FAST to detect keypoints.
2. **Orientation:** Compute orientation of each keypoint using intensity centroid:

$$\theta = \tan^{-1}\left(\frac{m_{01}}{m_{10}}\right)$$

   where

$$m_{pq} = \sum_x \sum_y x^p y^q I(x, y)$$

3. **Description:** Use a rotated version of BRIEF to generate binary descriptors invariant to rotation.

Advantages:

- Rotation and scale invariant.
- Fast and efficient (used in OpenCV's feature matching).
- Binary descriptors are compact and fast to compare.

Disadvantages:

- Less accurate for very large scale changes.
- Sensitive to significant illumination variations.

7.5 Comparison of Corner Detection Methods

Method	Speed	Rotation Invariant	Scale Invariant	Descriptor Provided
Harris	Medium	Yes	No	No
Shi-Tomasi	Medium	Yes	No	No
FAST	Very High	No	No	No
ORB	High	Yes	Partial	Yes (BRIEF)

Conclusion:

- **Harris** and **Shi-Tomasi** are reliable for static, small-scale tasks.
- **FAST** is preferred for real-time applications.
- **ORB** combines detection and description efficiently, making it a practical choice for modern computer vision systems like SLAM, AR, and object tracking.

8 Harris Corner Detection (With Example)

The Harris Corner Detector (Chris Harris and Mike Stephens, 1988) is a widely used method for detecting corner points in an image. Corners are points where the image intensity changes significantly in multiple directions. They are crucial for tracking, matching, and 3D reconstruction.

8.1 Mathematical Foundation

1. Intensity Change Function: For a small shift (u, v) around a pixel (x, y):

$$E(u,v) = \sum_{x,y} w(x,y)\,[I(x+u, y+v) - I(x,y)]^2$$

where $w(x, y)$ is a Gaussian window function that gives more weight to the center pixels.

2. Taylor Series Approximation: Expanding $I(x + u, y + v)$ using the first-order Taylor series:

$$I(x+u, y+v) \approx I(x,y) + I_x u + I_y v$$

where I_x and I_y are the image gradients along x and y directions.

Substituting gives:

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$$

where

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

is the **second moment matrix** (or structure tensor).

3. Eigenvalue Analysis: Let λ_1, λ_2 be the eigenvalues of M.

- Flat region: $\lambda_1 \approx 0, \lambda_2 \approx 0$
- Edge: one large, one small eigenvalue ($\lambda_1 \gg \lambda_2$)
- Corner: both large (λ_1 and λ_2 large)

4. Harris Corner Response Function: Instead of computing eigenvalues directly, Harris proposed:

$$R = \det(M) - k \cdot (\text{trace}(M))^2$$

where k is an empirical constant (typically 0.04–0.06).

- If $R > 0 \rightarrow$ corner
- If $R < 0 \rightarrow$ edge
- If $R \approx 0 \rightarrow$ flat region

8.2 Algorithm Steps

1. Convert image to grayscale.
2. Compute gradients I_x and I_y using Sobel filters.
3. Compute products of derivatives: I_x^2, I_y^2, and $I_x I_y$.
4. Apply Gaussian filter to each product to reduce noise.
5. Compute the Harris response R for each pixel.
6. Threshold R to detect corner points.
7. Apply non-maximum suppression to keep only strongest corners.

8.3 Worked Example

Consider the following 3×3 grayscale image patch:

$$I = \begin{bmatrix} 100 & 100 & 100 \\ 100 & 150 & 200 \\ 100 & 200 & 250 \end{bmatrix}$$

Step 1: Compute Gradients (Sobel Operator)

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Compute gradients at the center pixel (2,2):

$$I_x = (-1)(100) + 0(150) + 1(200) + (-2)(100) + 0(150) + 2(200) + (-1)(100) + 0(150) + 1(250)$$

$$I_x = (-100 + 200 - 200 + 400 - 100 + 250) = 450$$

$$I_y = (-1)(100) + (-2)(100) + (-1)(100) + 0(100) + 0(150) + 0(200) + 1(100) + 2(200) + 1(250)$$

$$I_y = (-100 - 200 - 100 + 100 + 400 + 250) = 350$$

Step 2: Compute Products of Derivatives

$$I_x^2 = 450^2 = 202{,}500$$

$$I_y^2 = 350^2 = 122{,}500$$

$$I_x I_y = 450 \times 350 = 157{,}500$$

Step 3: Construct Structure Tensor M

$$M = \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} = \begin{bmatrix} 202{,}500 & 157{,}500 \\ 157{,}500 & 122{,}500 \end{bmatrix}$$

Step 4: Compute Corner Response R

$$\det(M) = (202{,}500)(122{,}500) - (157{,}500)^2 = 24.8 \times 10^9 - 24.8 \times 10^9 = 0$$

$$\text{trace}(M) = 202{,}500 + 122{,}500 = 325{,}000$$

$$R = \det(M) - k \cdot (\text{trace}(M))^2 = 0 - 0.04 \times (325{,}000)^2 = -4.2 \times 10^{10}$$

Interpretation: $R < 0 \rightarrow$ indicates an edge rather than a true corner at this pixel.
Now, consider a modified region where both directions vary strongly:

$$I = \begin{bmatrix} 100 & 150 & 200 \\ 150 & 200 & 250 \\ 200 & 250 & 300 \end{bmatrix}$$

Gradients at center are higher and more balanced, producing:

$$I_x \approx 600, \quad I_y \approx 600$$

$$I_x^2 = 360{,}000, \quad I_y^2 = 360{,}000, \quad I_x I_y = 360{,}000$$

$$M = \begin{bmatrix} 360{,}000 & 360{,}000 \\ 360{,}000 & 360{,}000 \end{bmatrix}$$

$$\det(M) = (360{,}000)^2 - (360{,}000)^2 = 0$$

To simulate realistic variation, if small asymmetry exists:

$$M = \begin{bmatrix} 400{,}000 & 350{,}000 \\ 350{,}000 & 420{,}000 \end{bmatrix}$$

$$\det(M) = 400{,}000 \times 420{,}000 - 350{,}000^2 = 168 \times 10^9 - 122.5 \times 10^9 = 45.5 \times 10^9$$

$$\text{trace}(M) = 820{,}000$$

$$R = 45.5 \times 10^9 - 0.04 \times (820{,}000)^2 = 45.5 \times 10^9 - 26.9 \times 10^9 = 18.6 \times 10^9$$

Since $R > 0$, the pixel represents a **corner**.

8.4 Visualization of Responses

- **Flat Region:** $R \approx 0$
- **Edge:** $R < 0$
- **Corner:** $R > 0$

$$\text{Example Classification:} \quad \begin{bmatrix} \text{Flat} & \text{Edge} & \text{Corner} \\ \text{Edge} & \text{Corner} & \text{Edge} \\ \text{Flat} & \text{Edge} & \text{Corner} \end{bmatrix}$$

8.5 Advantages

- Robust to rotation.
- Precise localization of corners.
- Good performance on textured regions.

8.6 Disadvantages

- Not scale-invariant (fails when image is zoomed or resized).
- Sensitive to parameter k and thresholding.

8.7 Conclusion

The Harris detector measures the intensity change in all directions, making it an effective and computationally efficient method for detecting corners. In the example, corners were identified where gradients changed strongly in both x and y directions, confirming the algorithm's principle of multi-directional variation.

9 Scale-Invariant Feature Transform (SIFT)

The Scale-Invariant Feature Transform (SIFT) algorithm, proposed by David G. Lowe in 1999 and refined in 2004, is one of the most powerful feature detection and description methods in computer vision. SIFT detects stable keypoints in an image and computes distinctive feature descriptors that are invariant to scale, rotation, and moderate changes in illumination or viewpoint.

9.1 Key Properties of SIFT

- **Scale-invariant:** Detects the same keypoints regardless of image zoom.
- **Rotation-invariant:** Descriptor orientation compensates for rotation.
- **Partially affine-invariant:** Handles small viewpoint changes.
- **Robust to illumination:** Uses normalized gradients instead of intensities.

9.2 Stages of the SIFT Algorithm

1. Scale-Space Construction

SIFT constructs a scale-space representation of the image to detect keypoints invariant to scale. The scale-space of an image $I(x, y)$ is defined as:

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

where $G(x, y, \sigma)$ is a Gaussian function:

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

The image is progressively blurred with increasing σ, generating a set of images at different scales (octaves).

2. Difference of Gaussians (DoG)

Keypoints are detected using the Difference of Gaussians (DoG) between adjacent scales:

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

This approximates the Laplacian of Gaussian (LoG) but is computationally faster. Local extrema in $D(x, y, \sigma)$ (in 3D: across x, y, σ) correspond to potential keypoints.

3. Keypoint Localization

For each candidate keypoint, SIFT fits a 3D quadratic function to local DoG values to refine its position, scale, and contrast. A keypoint is rejected if:

- Its contrast is below a threshold.
- It lies along an edge (unstable localization).

The Taylor expansion around candidate (x, y, σ) is:

$$D(\mathbf{x}) = D + \frac{\partial D}{\partial \mathbf{x}}^T \mathbf{x} + \frac{1}{2}\mathbf{x}^T \frac{\partial^2 D}{\partial \mathbf{x}^2}\mathbf{x}$$

where $\mathbf{x} = (x, y, \sigma)^T$.

4. Orientation Assignment

Each keypoint is assigned an orientation based on local image gradients in its neighborhood:

$$m(x, y) = \sqrt{(L_x)^2 + (L_y)^2}, \quad \theta(x, y) = \tan^{-1}\left(\frac{L_y}{L_x}\right)$$

An orientation histogram (36 bins for 360°) is built around the keypoint. The dominant orientation defines the keypoint's rotation, ensuring rotation invariance.

5. Keypoint Descriptor Formation

A 16×16 region around each keypoint is divided into sixteen 4×4 subregions. For each subregion, an 8-bin orientation histogram is computed, resulting in a 128-dimensional descriptor vector:

$$128 = 4 \times 4 \times 8$$

To enhance robustness:

- The descriptor vector is normalized to unit length.
- Values are thresholded (typically capped at 0.2) and re-normalized to reduce illumination effects.

9.3 Simplified Example (Conceptual Demonstration)

Consider a 5×5 grayscale image patch centered at a corner-like structure:

$$I = \begin{bmatrix} 50 & 55 & 60 & 65 & 70 \\ 55 & 60 & 70 & 80 & 85 \\ 60 & 70 & 90 & 110 & 120 \\ 65 & 80 & 110 & 130 & 140 \\ 70 & 90 & 120 & 140 & 150 \end{bmatrix}$$

Step 1: Scale-space Construction

Apply Gaussian blurs with increasing $\sigma = \{1, 2, 4\}$, generating progressively smoother images. At higher σ, fine details disappear while major structures remain.

Step 2: Difference of Gaussians (DoG)

Compute:

$$D(x, y, \sigma_1) = L(x, y, 2) - L(x, y, 1), \quad D(x, y, \sigma_2) = L(x, y, 4) - L(x, y, 2)$$

Suppose at the central pixel (3,3), we get:

$$D(3, 3, 1) = 18, \quad D(3, 3, 2) = -15$$

Since (3,3) is an extremum (maximum or minimum) across scales and space, it's a candidate keypoint.

Step 3: Gradient and Orientation

Compute gradient magnitudes and orientations in a local 3×3 neighborhood.

At (3,3):

$$L_x = \frac{L(3,4) - L(3,2)}{2} = \frac{110 - 70}{2} = 20$$

$$L_y = \frac{L(4,3) - L(2,3)}{2} = \frac{110 - 70}{2} = 20$$

$$m = \sqrt{20^2 + 20^2} = 28.3, \quad \theta = 45^\circ$$

Thus, the keypoint is assigned an orientation of 45°.

Step 4: Descriptor Generation

For simplicity, consider a 4×4 region around the keypoint. Local gradient directions (quantized into 8 bins) and magnitudes are used to build histograms per subregion.

Example histogram (conceptual):

Subregion	Dominant Angle (°)	Magnitude
1	0	10
2	45	15
3	90	12
4	45	18

The 16 subregion histograms (each 8 bins) are concatenated and normalized → 128D vector.

This vector is the **SIFT descriptor** for that keypoint, robust to scale and rotation.

9.4 SIFT Keypoint Visualization

Detected keypoints are typically displayed as circles:

- Center = keypoint location.
- Radius = scale.
- Arrow = orientation.

9.5 Advantages of SIFT

- Highly distinctive and robust to noise, scale, and rotation.
- Effective for matching across images with different views.
- Works well even under partial occlusion.

9.6 Disadvantages of SIFT

- Computationally intensive compared to ORB or FAST.
- Patent restrictions (prior to 2020) limited its open-source use.
- Performance degrades for drastic illumination or affine distortions.

9.7 Conclusion

SIFT revolutionized local feature detection by introducing a mathematically sound method for identifying keypoints that are invariant to scale and rotation. In the simplified example, the central region with strong multi-directional gradients produced a stable keypoint. Its descriptor captured local orientation patterns, enabling robust matching between different images of the same scene or object.

10 Texture Analysis

Texture refers to the spatial arrangement of pixel intensities that represent the surface characteristics or patterns of an image. It plays a critical role in image classification, segmentation, and pattern recognition. Two commonly used methods for quantitative texture analysis are the Gray Level Co-occurrence Matrix (GLCM) and the Local Binary Pattern (LBP).

10.1 Gray Level Co-occurrence Matrix (GLCM)

The GLCM is a statistical method that examines the spatial relationship between pixels of specific gray levels within an image. It represents how often pairs of pixel values (gray levels) occur at a certain distance and direction from each other.

Mathematical Definition

Given a grayscale image with gray levels $0, 1, 2, ..., N-1$, the GLCM $P(i, j)$ is defined as:

$$P(i, j, d, \theta) = \text{number of times two pixels with values } i \text{ and } j \text{ occur at distance } d \text{ and direction } \theta$$

Common directions are:

$$0^\circ \text{ (horizontal)}, \quad 45^\circ, \quad 90^\circ \text{ (vertical)}, \quad 135^\circ$$

After computing, $P(i, j)$ is often normalized by dividing by the total number of pairs:

$$p(i, j) = \frac{P(i, j)}{\sum_i \sum_j P(i, j)}$$

Example

Consider a simple 4×4 grayscale image with intensity levels 0–2:

$$I = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 2 & 2 & 1 \\ 1 & 2 & 0 & 0 \\ 2 & 2 & 1 & 0 \end{bmatrix}$$

Compute the GLCM for distance $d = 1$ and direction 0° (horizontal neighbors).
We count all horizontal pairs (i, j):

Pair (i,j)	Count
(0,0)	1
(0,1)	1
(1,1)	1
(0,2)	1
(1,2)	1
(2,0)	2
(2,1)	1
(2,2)	2

From this, construct the 3×3 GLCM matrix:

$$P = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 2 & 1 & 2 \end{bmatrix}$$

Normalize it:

$$p(i, j) = \frac{P(i, j)}{\sum P(i, j)} = \frac{P(i, j)}{9}$$

Texture Features from GLCM

From the normalized matrix $p(i, j)$, we can compute several texture measures:

- **Contrast:**
$$\text{Contrast} = \sum_{i,j} (i-j)^2 p(i,j)$$
High contrast indicates large intensity differences.

- **Correlation:**
$$\text{Correlation} = \sum_{i,j} \frac{(i-\mu_i)(j-\mu_j)p(i,j)}{\sigma_i \sigma_j}$$
Measures linear dependency between pixel pairs.

- **Energy (Angular Second Moment):**
$$\text{Energy} = \sum_{i,j} [p(i,j)]^2$$
High energy indicates a uniform or periodic texture.

- **Homogeneity (Inverse Difference Moment):**
$$\text{Homogeneity} = \sum_{i,j} \frac{p(i,j)}{1+|i-j|}$$
High homogeneity means pixel pairs are similar.

Example Calculation: Contrast

Using the normalized $p(i, j)$:

$$\text{Contrast} = \sum (i-j)^2 p(i,j) = (0-1)^2 \times \frac{1}{9} + (0-2)^2 \times \frac{1}{9} + (2-0)^2 \times \frac{2}{9} + ...$$

$$\text{Contrast} \approx 1.78$$

Hence, the texture shows moderate contrast (visible transitions between gray levels).

10.2 Local Binary Pattern (LBP)

The Local Binary Pattern (LBP) is a texture descriptor that encodes the relationship between a pixel and its neighboring pixels into a binary number. It is computationally simple and highly effective for texture classification, face recognition, and surface inspection.

Mathematical Definition

For a pixel at position (x_c, y_c) with intensity I_c, and its surrounding P neighbors with intensities I_p, the LBP code is defined as:

$$LBP_{P,R} = \sum_{p=0}^{P-1} s(I_p - I_c) \times 2^p$$

where

$$s(x) = \begin{cases} 1, & \text{if } x \geq 0 \\ 0, & \text{if } x < 0 \end{cases}$$

R is the radius of the circular neighborhood.

Example (3×3 neighborhood)

$$\begin{bmatrix} 40 & 30 & 20 \\ 45 & 50 & 25 \\ 60 & 55 & 35 \end{bmatrix}$$

Central pixel $I_c = 50$.

Compare each neighbor I_p with I_c:

Position	I_p	$I_p - I_c$	$s(I_p - I_c)$
(−1,−1)	40	−10	0
(−1,0)	30	−20	0
(−1,1)	20	−30	0
(0,1)	25	−25	0
(1,1)	35	−15	0
(1,0)	55	+5	1
(1,−1)	60	+10	1
(0,−1)	45	−5	0

Concatenate the binary pattern clockwise (starting top-left):

Binary: 00001110

Convert to decimal:

$$LBP = (00001110)_2 = 14$$

So, the local binary pattern value for the central pixel is **14**.

Rotation-Invariant LBP

To achieve rotation invariance, the binary pattern is circularly rotated to get the minimum possible value:

$$LBP_{ri} = \min(\text{rotate}(LBP))$$

Texture Histogram

LBP values for all pixels form a histogram representing the texture of the image. This histogram serves as a feature vector for classification or comparison between textures.

10.3 Comparison of GLCM and LBP

Property	GLCM	LBP
Type	Statistical (2nd order)	Structural (local pattern)
Output	Co-occurrence matrix & derived features	Binary codes & histogram
Invariance	Direction, limited scale invariance	Rotation and illumination invariant
Computational Cost	Moderate to high	Low
Applications	Remote sensing, medical imaging	Face recognition, defect detection

10.4 Conclusion

Both GLCM and LBP are foundational texture analysis techniques. GLCM captures global texture relationships through co-occurrence statistics, while LBP captures local texture patterns through binary thresholding. Combining both can yield powerful hybrid texture descriptors for robust image classification and segmentation tasks.

11 Binary Shape Analysis

Binary Shape Analysis involves extracting quantitative and structural information from binary (black-and-white) images. These images typically represent segmented objects, where pixel values are 1 (object) or 0 (background). Analyzing binary shapes helps in object recognition, counting, classification, and geometric measurement.

11.1 Connectedness

Connectedness defines which neighboring pixels belong to the same object. In a binary image, a pixel can be connected to its neighbors in one of two main ways:

- **4-connectivity:** Each pixel is connected to its horizontal and vertical neighbors.

$$N_4(p) = \{(x \pm 1, y), (x, y \pm 1)\}$$

- **8-connectivity:** Each pixel is connected to all surrounding neighbors (horizontal, vertical, and diagonal).

$$N_8(p) = \{(x + i, y + j) \mid i, j \in \{-1, 0, 1\}, (i, j) \neq (0, 0)\}$$

Example:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Here, the central pixel is connected to all four neighbors → 4-connected structure. If diagonals are considered, it becomes 8-connected.

11.2 Object Labelling and Counting

Connected Component Labeling (CCL) assigns a unique label to each connected object in a binary image, allowing objects to be counted and analyzed separately.

Algorithm (Two-pass method)

1. **First pass:** Scan the image left-to-right, top-to-bottom. Assign temporary labels to connected foreground pixels using neighborhood analysis.
2. **Equivalence resolution:** Merge equivalent labels (i.e., those connected through different paths).
3. **Second pass:** Replace temporary labels with final unique labels.

Example:

$$\begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

→ produces two distinct labeled objects.

$$\text{Labelled Output: } \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 2 \\ 0 & 0 & 2 & 2 \end{bmatrix}$$

→ Object count = 2.

11.3 Distance Functions

A distance function assigns each pixel a value proportional to its distance from the nearest object boundary (or background). It is fundamental for computing shape descriptors, skeletons, and morphological operations. Common distance measures:

- **Euclidean:** $D_E(p,q) = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2}$
- **City Block (Manhattan):** $D_4(p,q) = |x_p - x_q| + |y_p - y_q|$
- **Chessboard:** $D_8(p,q) = \max(|x_p - x_q|, |y_p - y_q|)$

Example: For a binary object:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The Euclidean distance transform assigns each foreground pixel a value = distance to nearest 0 pixel (boundary).

$$D_E \approx \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1.4 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

11.4 Skeletons and Thinning

The skeleton (or medial axis) represents the central structure of a binary object, maintaining its topology but reducing thickness to a single-pixel width. Thinning algorithms are iterative methods to obtain the skeleton.

Skeleton Definition

For an object A and structuring element B:

$$S(A) = \bigcup_{k=0}^{K} [(A \ominus kB) - ((A \ominus kB) \circ B)]$$

where:

- $\ominus$: morphological erosion
- $\circ$: morphological opening

Example: A binary cross shape reduces to a single central pixel skeleton.

Applications:

- Fingerprint ridge thinning.
- Road network extraction.
- Shape topology analysis.

11.5 Contour Properties

The contour of a binary object is the set of pixels that form its boundary. Analyzing contour properties provides geometric shape descriptors. Common Properties:

- **Perimeter (P):** Number of boundary pixels.
- **Compactness:** $C = \frac{P^2}{4\pi A}$
- **Eccentricity:** Ratio of major to minor axis of the fitted ellipse.
- **Circularity:** Circ $= \frac{4\pi A}{P^2}$

Example: If a binary object has $A = 100$ pixels and $P = 40$:

$$\text{Circularity} = \frac{4\pi(100)}{40^2} = 0.785$$

A perfect circle has circularity = 1.

11.6 Hu Moments

Hu Moments (proposed by Ming-Kuei Hu, 1962) are invariant image moments derived from normalized central moments. They are invariant to translation, scale, and rotation — making them powerful for object recognition.

Mathematical Definition

Raw moments:

$$M_{pq} = \sum_x \sum_y x^p y^q f(x, y)$$

Centroid:

$$\bar{x} = \frac{M_{10}}{M_{00}}, \quad \bar{y} = \frac{M_{01}}{M_{00}}$$

Central moments:

$$\mu_{pq} = \sum_x \sum_y (x - \bar{x})^p (y - \bar{y})^q f(x, y)$$

Normalized central moments:

$$\eta_{pq} = \frac{\mu_{pq}}{(\mu_{00})^{(1+\frac{p+q}{2})}}$$

From these, Hu defined 7 invariant moments $(\phi_1, \ldots, \phi_7)$ such as:

$$\phi_1 = \eta_{20} + \eta_{02}, \quad \phi_2 = (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2$$

and so on.

Example: For a simple filled square, ϕ_1 and ϕ_2 values are characteristic and identical even after rotation or scaling.

11.7 Area and Perimeter

These are the most basic shape descriptors:

- **Area (A):** Number of foreground pixels.

$$A = \sum_{x,y} f(x, y)$$

- **Perimeter (P):** Number of pixels on the boundary (using 4- or 8-connectivity).

Example:

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Area $A = 6$, Perimeter $P = 10$.

Derived measure:

$$\text{Compactness} = \frac{P^2}{A} = \frac{10^2}{6} = 16.67$$

$\rightarrow$ less compact (elongated) shape.

11.8 Conclusion

Binary shape analysis provides quantitative shape descriptors that allow for comparison and classification of objects. Connectedness and labeling identify separate components, distance functions and skeletons reveal structure, and contour- or moment-based measures capture geometric properties. Together, these form the foundation of object recognition, pattern analysis, and image morphology.

12 Image Segmentation

Image Segmentation is the process of partitioning an image into meaningful regions or objects. It simplifies image analysis by grouping pixels that share similar characteristics such as color, intensity, or texture. Segmentation is fundamental to object detection, recognition, and tracking in computer vision. This section discusses four major segmentation methods: Watershed, Graph Cut, Region Growing, and K-Means Clustering.

12.1 Watershed Segmentation

The Watershed algorithm is inspired by topography, where the image is treated as a landscape and intensity values represent elevation. Watershed lines are the boundaries separating different catchment basins formed by flooding the image from its minima.

Mathematical Concept

Given an image $I(x, y)$, consider the gradient magnitude:

$$G(x, y) = \sqrt{(I_x)^2 + (I_y)^2}$$

Regions of low gradient correspond to homogeneous areas, and high gradients represent edges.

Algorithm Steps:

1. Compute the gradient image $G(x, y)$.
2. Identify local minima (seed points).
3. Simulate flooding from each minimum.
4. As water basins meet, construct **watershed lines** at their boundaries.

Example: For an image with two bright circular objects on a dark background, watershed segmentation can separate the objects even if they slightly touch.

Mathematical Expression:

$$W(I) = \text{watershed transform of } I(x, y)$$

If M_i are the catchment basins and B_{ij} the boundaries:

$$I(x, y) = \bigcup_i M_i \cup \bigcup_{i,j} B_{ij}$$

Advantages:

- Accurate edge localization.
- Produces closed and continuous boundaries.

Disadvantages:

- Sensitive to noise and over-segmentation.

12.2 Graph-Cut Segmentation

Graph Cut formulates image segmentation as a graph partitioning problem. Each pixel is a node, and edges connect neighboring pixels. Segmentation is achieved by minimizing an energy function that balances data fidelity and smoothness.

Mathematical Model

Let $G = (V, E)$ be a graph where:

- V: set of pixels (nodes)
- E: set of edges connecting neighboring pixels

Each edge has a weight w_{ij} representing similarity between pixels i and j.
The segmentation is obtained by minimizing:

$$E(L) = \sum_i D_i(L_i) + \sum_{(i,j)\in E} V_{ij}(L_i, L_j)$$

where:

- $D_i(L_i)$: data term (how well pixel i fits label L_i)
- V_{ij}: smoothness term encouraging neighboring pixels to share labels

Algorithm Steps:

1. Build a weighted graph with nodes for pixels and two terminals: source (object) and sink (background).
2. Assign edge weights based on color or intensity similarity.
3. Compute the minimum cut that separates source and sink while minimizing energy $E(L)$.

Example: For an image of a person on a plain background, user-defined "foreground" and "background" seeds guide the graph cut to produce accurate segmentation.

Advantages:

- Produces globally optimal segmentation.
- Handles weak boundaries and noise effectively.

Disadvantages:

- Computationally expensive.
- May require user input for seeds.

12.3 Region Growing

Region Growing is a pixel-based segmentation method that groups neighboring pixels based on similarity in intensity, color, or texture starting from seed points.

Algorithm Steps:

1. Select initial seed pixels (manually or automatically).
2. For each seed, compare neighboring pixels:

$$|I(x, y) - I(s_x, s_y)| < T$$

   If true, the pixel joins the region.

3. Continue until no new pixels meet the criteria.

Example: Consider a grayscale image region:

$$I = \begin{bmatrix} 45 & 46 & 50 & 180 \\ 47 & 49 & 52 & 182 \\ 44 & 46 & 55 & 185 \end{bmatrix}$$

Starting with seed = 47 and threshold $T = 10$, pixels 45–55 form one region, and pixels 180–185 form another.

Advantages:

- Simple and intuitive.
- Produces connected and homogeneous regions.

Disadvantages:

- Sensitive to noise and threshold selection.
- Requires good seed initialization.

12.4 K-Means Segmentation

K-Means clustering is an unsupervised method that partitions image pixels into k clusters based on feature similarity (e.g., color or intensity).

Mathematical Formulation

Given pixels $\{x_1, x_2, \ldots, x_n\}$ and cluster centers $\{c_1, c_2, \ldots, c_k\}$, minimize the total intra-cluster variance:

$$J = \sum_{i=1}^{k} \sum_{x_j \in S_i} \|x_j - c_i\|^2$$

where S_i is the set of pixels assigned to cluster i.

Algorithm Steps:

1. Choose k initial cluster centers randomly.
2. Assign each pixel to the nearest cluster:

$$\text{Assign } x_j \text{ to } c_i \text{ if } \|x_j - c_i\| \text{ is minimum.}$$

3. Update cluster centers:

$$c_i = \frac{1}{|S_i|} \sum_{x_j \in S_i} x_j$$

4. Repeat until convergence.

Example:

For a grayscale image with pixel intensities:

$$[30, 35, 32, 160, 170, 180]$$

and $k = 2$, initial centers $c_1 = 35, c_2 = 170$:

$$\text{Cluster 1: } [30, 32, 35], \quad \text{Cluster 2: } [160, 170, 180]$$

Updated centers:

$$c_1 = 32.3, \quad c_2 = 170$$

$\rightarrow$ Two clusters representing dark and bright regions.

Advantages:

- Fast and simple to implement.
- Effective for well-separated clusters.

Disadvantages:

- Sensitive to initial cluster centers.
- Assumes spherical clusters; may fail on complex textures.

12.5 Comparison of Segmentation Methods

| Method | Type | Strengths | Weaknesses | |———|——|————|————-| | Watershed | Gradient-based | Precise boundaries | Over-segmentation, noise-sensitive | | Graph Cut | Energy minimization | Global optimum, robust | Computationally intensive | | Region Growing | Region-based | Simple, connected regions | Seed and threshold dependent | | K-Means | Clustering | Fast, unsupervised | Sensitive to initialization |

12.6 Conclusion

Different segmentation methods suit different image characteristics. Watershed and Graph Cut are edge- and energy-based, producing precise contours. Region Growing and K-Means are region-based, providing homogeneous segmentations. In practice, hybrid approaches (e.g., K-Means + Watershed) are often used for more robust and accurate segmentation results.

13 Boundary Pattern Analysis

Boundary Pattern Analysis involves representing and describing the boundary or contour of objects in a digital image. It provides compact and invariant descriptions of object shapes for recognition and classification. Two widely used techniques for boundary representation are the Chain Code and the Polygonal Approximation.

13.1 Chain Code Representation

The Chain Code (proposed by Freeman, 1961) encodes the boundary of an object as a sequence of directional movements between connected boundary pixels. It provides a simple, translation-invariant representation of the boundary pattern.

Concept:

Given a binary object, trace its boundary in either 4- or 8-connectivity, and record the direction of each step.

Direction Encoding:

$$\begin{matrix} & 3 & 2 & 1 \\ 4 & & & 0 \\ 5 & 6 & 7 & \end{matrix}$$

4-connectivity directions:

$$0 : \text{right}, \quad 1 : \text{up}, \quad 2 : \text{left}, \quad 3 : \text{down}$$

8-connectivity directions:

$$0, 1, 2, \ldots, 7 \text{ correspond to movements at } 45^\circ \text{ increments.}$$

Algorithm Steps:

1. Find the starting pixel on the object boundary (e.g., topmost-leftmost pixel).
2. Choose connectivity (4 or 8).
3. Traverse the boundary clockwise (or counterclockwise).
4. Record the direction of each move.
5. Stop when the traversal returns to the starting pixel.

Example: For a binary object:

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

The boundary trace (8-connectivity) may produce the chain code:

$$C = [0, 6, 6, 4, 2, 2, 0]$$

Normalization (Rotation Invariance): The chain code can be made rotation-invariant by using the **first difference chain code**:

$$C' = (c_{i+1} - c_i) \mod N$$

where N is 4 or 8 depending on connectivity.

Advantages:

- Compact representation.
- Easy to reconstruct the boundary.
- Useful for shape comparison.

Disadvantages:

- Sensitive to noise and quantization.
- Dependent on starting point.

Reconstruction:

Given a starting point (x_0, y_0) and chain code C, boundary points can be reconstructed using:

$$(x_{i+1}, y_{i+1}) = (x_i, y_i) + \Delta(C_i)$$

where $\Delta(C_i)$ is the displacement vector for the code direction.

—

13.2 Polygonal Approximation

Polygonal Approximation represents the boundary as a set of straight line segments that approximate its shape. This reduces data redundancy and simplifies complex contours while preserving essential geometric information.

Concept:

A continuous boundary can be approximated by connecting a subset of its points to form a polygon. This is useful for object simplification, feature extraction, and shape matching.

Mathematical Definition:

Let the boundary be represented by ordered points:

$$P = \{p_1, p_2, \dots, p_n\}$$

Polygonal approximation seeks a reduced set of vertices:

$$Q = \{q_1, q_2, \dots, q_m\}, \quad m < n$$

that minimizes deviation from the original curve while maintaining shape fidelity.

Error Criterion:

$$E = \max_{p_i \in P} d(p_i, \overline{q_j q_{j+1}})$$

where $d(p_i, \overline{q_j q_{j+1}})$ is the perpendicular distance from boundary point p_i to line segment $\overline{q_j q_{j+1}}$.

Algorithm (Ramer–Douglas–Peucker):

1. Connect the first and last boundary points by a straight line.
2. Find the point with the maximum perpendicular distance from the line.
3. If the distance $> \varepsilon$ (tolerance), keep that point and recursively process the two resulting segments.
4. Stop when all distances are less than ε.

Example: Suppose a boundary curve has points:

$$P = \{(0,0), (1,1), (2,2), (3,1), (4,0)\}$$

For tolerance $\varepsilon = 0.5$, the approximation keeps only:

$$Q = \{(0,0), (2,2), (4,0)\}$$

→ A triangle-like simplified representation.

Advantages:

- Reduces data storage.
- Preserves overall shape structure.
- Facilitates shape matching and comparison.

Disadvantages:

- Sensitive to noise and ε value.
- Over-simplification may lose shape details.

13.3 Comparison of Boundary Representation Methods

Method	Representation Type	Data Reduction	Invariance	Typical Use
Chain Code	Symbolic (directional)	Moderate	Translation, rotation (after normalization)	Boundary tracing and recognition
Polygonal Approximation	Geometric (vertex-based)	High	Translation, rotation, scale	Shape simplification, object modeling

13.4 Conclusion

Boundary pattern analysis is essential for compact and robust shape representation. The Chain Code captures local directional information along the contour, while Polygonal Approximation provides a simplified geometric model. Together, they enable efficient object description, comparison, and classification in shape-based computer vision applications.